



와이어샤크를 통한 프로토콜 분석

이 프레젠테이션에서는 와이어샤크(Wireshark)를 사용하여 다양한 네트워크 프로토콜을 심층적으로 분석합니다. 패킷 캡처 파일을 통해 IPv4/IPv6 단편화, ICMP, UDP, TCP 연결 수립 과정 등을 살펴보고 이론적 개념을 실제 패킷 데이터로 확인합니다.

와이어샤크를 통한 프로토콜 분석(1)

학습 목표

- 와이어샤크를 사용한 패킷 캡처 파일 분석 방법 이해
- 이론으로 학습한 네트워크 프로토콜의 실제 작동 원리 확인
- 프로토콜 헤더 필드의 의미와 역할 파악

이 장에서는 GitHub에 공개된 패킷 캡처 파일을 활용하여 분석을 진행합니다:

<https://github.com/kangtegong/self-learning-cs2>

```
ipv4-fragmentation.pcapng  
ipv6-fragmentation.pcapng  
3-way-handshake.pcapng  
connection-close.pcapng  
tcp-retransmission.pcapng  
http_request_response.pcapng
```

이 파일들로 실습해 봅시다!



IPv4 단편화 + ICMP 분석

ipv4-fragmentation 파일 개요

이 패킷 캡처 파일은 IPv4의 단편화 과정과 ICMP 패킷 교환을 보여줍니다:

- 1번~7번 패킷: 호스트 10.0.0.1이 10.0.0.2에게 패킷 전송
- 8번~14번 패킷: 호스트 10.0.0.2가 10.0.0.1에게 패킷 전송

단편화는 큰 데이터가 네트워크의 최대 전송 단위(MTU)를 초과할 때 발생하는 현상으로, 하나의 큰 패킷이 여러 개의 작은 패킷으로 나뉘어 전송됩니다.

No.	Time	Source	Destination	Protocol	Length
1	0.000000	10.0.0.1	10.0.0.2	IPv4	1514
2	0.000000	10.0.0.1	10.0.0.2	IPv4	1514
3	0.000000	10.0.0.1	10.0.0.2	IPv4	1514
4	0.000000	10.0.0.1	10.0.0.2	IPv4	1514
5	0.000000	10.0.0.1	10.0.0.2	IPv4	1514
6	0.000000	10.0.0.1	10.0.0.2	IPv4	1514
7	0.000000	10.0.0.1	10.0.0.2	ICMP	134
8	0.070025	10.0.0.2	10.0.0.1	IPv4	1514
9	0.070025	10.0.0.2	10.0.0.1	IPv4	1514
10	0.070025	10.0.0.2	10.0.0.1	IPv4	1514
11	0.070025	10.0.0.2	10.0.0.1	IPv4	1514
12	0.070025	10.0.0.2	10.0.0.1	IPv4	1514
13	0.070025	10.0.0.2	10.0.0.1	IPv4	1514
14	0.070025	10.0.0.2	10.0.0.1	ICMP	134

와이어샤크에서 캡처된 IPv4 단편화 패킷

IPv4 패킷 분석 - 주소 정보

IPv4 헤더 필드

IPv4 헤더는 패킷의 주소 지정과 단편화에 필요한 다양한 필드 포함

- 버전(Version): IPv4를 나타내는 값 4
- 헤더 길이(Header Length): 일반적으로 20바이트
- 서비스 유형(Type of Service): 패킷의 우선순위
- 전체 길이(Total Length): 패킷의 전체 크기

주소 지정 필드

IPv4 패킷의 주소 지정에 관련된 필드:

- 송신지 주소(Source Address): 출발지 IP 주소
- 수신지 주소(Destination Address): 목적지 IP 주소
- 각 주소는 32비트(4바이트) 크기
- 점으로 구분된 10진수 형태로 표현 (예: 10.0.0.1)

```
✓ Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 1500
    Identification: 0x2c2e (11310)
  > 001. .... = Flags: 0x1, More fragments
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 255
    Protocol: ICMP (1)
    Header Checksum: 0x55f0 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.0.0.1
    Destination Address: 10.0.0.2
```

IPv6 주소 표기법

IPv6 주소의 기본 구조

IPv6 주소는 128비트로 구성되며, 기본적으로 32개의 16진수가 필요합니다:

- 8개 그룹으로 나뉨 (각 그룹 4개의 16진수)
- 콜론(:)으로 그룹 구분
- 예: 2001:0db8:85a3:0000:0000:8a2e:0370:7334

IPv6 주소 축약 규칙 1

각 그룹의 앞쪽에 나오는 0은 생략 가능:

- 0001 → 1
- 예: 2001:db8:85a3:0:0:8a2e:370:7334
- (0000 → 0으로 줄여쓰기)

IPv6 주소 축약 규칙 2

연속된 0으로 구성된 그룹은 '::'으로 대체 가능:

- 예: 2001:db8:85a3::8a2e:370:7334
- '::'는 주소에서 한 번만 사용 가능
- 20f4:c750:2f42:53df::11:0은 실제로 20f4:c750:2f42:53df:0000:0000:0011:0000의 축약형

20f4:c750:2f42:53df:0000:0000:0011:0000

2b f7:f750:2ffb:53df:0000:0000:0000:1001

20f4:c750:2f42:53df:0000:0000:0011:0000

20f4:c750:2f42:53df::11:0

2b f7:f750:2ffb:53df:0000:0000:0000:1001

2b f7:f750:2ffb:53df::1001

UDP 데이터그램 분석

UDP 헤더 구조

UDP(User Datagram Protocol)는 단순한 헤더 구조를 가진 비연결형 전송 프로토콜입니다:

- 헤더 크기: 8바이트
- 송신지 포트(Source Port): 2바이트
- 수신지 포트(Destination Port): 2바이트
- 길이(Length): 헤더와 데이터를 포함한 전체 UDP 데이터그램 길이(바이트)
- 체크섬(Checksum): 오류 검출을 위한 필드

캡처 파일의 UDP 패킷 분석

1번과 2번 패킷의 UDP 헤더 정보:

- 송신지 포트와 수신지 포트 확인
- UDP 데이터그램 길이 확인
- UDP는 연결 설정 과정이 없어 TCP보다 오버헤드가 적음
- 신뢰성 보장 없이 빠른 전송에 중점

```
▼ User Datagram Protocol, Src Port: 58677, Dst Port: 58677
  Source Port: 58677
  Destination Port: 58677
  Length: 126
  Checksum: 0xeb4d [unverified]
  [Checksum Status: Unverified]
  [Stream index: 0]
  > [Timestamps]
  UDP payload (118 bytes)
```

TCP 연결 수립 분석

TCP의 연결 지향 특성

TCP(Transmission Control Protocol)는 연결 지향적 프로토콜로, 데이터 전송 전에 연결을 수립합니다:

- 신뢰성 있는 데이터 전송 보장
- 순서 보장 및 흐름 제어 제공
- 연결 수립과 종료 과정 필요

3-way-handshake 파일 분석

이 캡처 파일은 TCP 연결 수립 과정을 보여줍니다:

- 1번 패킷: 192.168.0.1이 10.10.10.1에게 연결 요청(SYN)
- 2번 패킷: 10.10.10.1이 192.168.0.1에게 확인 응답(SYN+ACK)
- 3번 패킷: 192.168.0.1이 10.10.10.1에게 확인 응답(ACK)

No.	Time	Source	Destination	Protocol
1	0.000000	192.168.0.1	10.10.10.1	TCP
2	0.307187	10.10.10.1	192.168.0.1	TCP
3	0.307372	192.168.0.1	10.10.10.1	TCP

TCP 패킷 분석 - 1단계(SYN)

1 TCP 헤더 구조

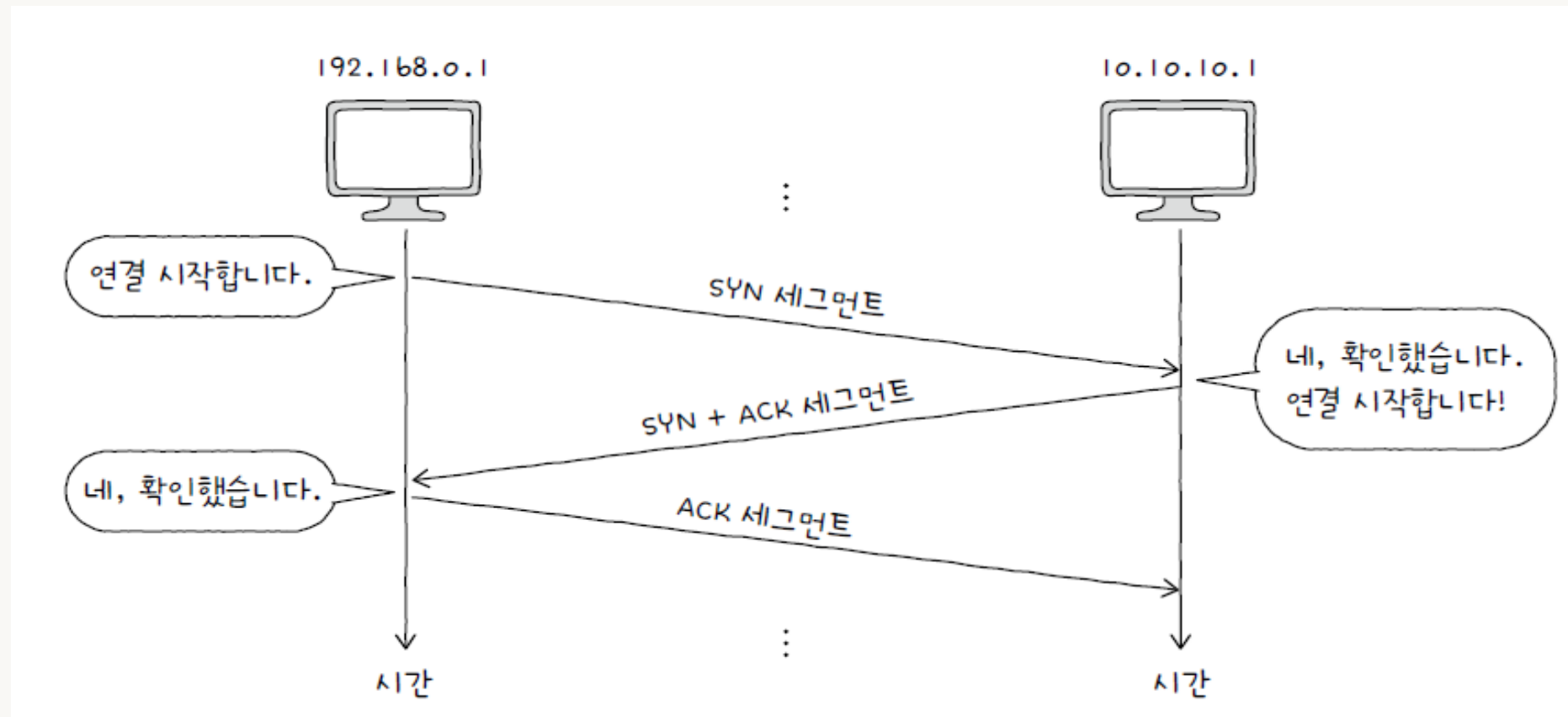
TCP 헤더는 연결 관리와 데이터 전송에 필요한 다양한 필드를 포함합니다:

- 포트 번호: 송신지/수신지 프로세스 식별
- 순서 번호: 데이터 바이트 스트림의 위치
- 확인 응답 번호: 다음에 받을 것으로 예상되는 순서 번호
- 플래그: 연결 상태와 제어 정보

2 첫 번째 패킷(SYN) 분석

클라이언트가 서버에게 보내는 연결 요청 패킷:

- 송신지 포트: 49859 (동적 포트 범위에서 임의 할당)
- 수신지 포트: 80 (HTTP 서비스 표준 포트)
- 순서 번호(raw): 3588415412
- SYN 플래그: 1 (연결 요청)



TCP 패킷 분석 - 2단계(SYN+ACK)

두 번째 패킷은 서버(10.10.10.1)가 클라이언트(192.168.0.1)에게 보내는 응답으로, 연결 요청을 수락하고 자신의 초기 순서 번호를 알립니다.



포트 번호 교환

첫 번째 패킷과 반대로 송신지/수신지 포트가 바뀜:

- 송신지 포트: 80 (서버)
- 수신지 포트: 49859 (클라이언트)



순서 번호 설정

서버의 초기 순서 번호 설정:

- 순서 번호(raw): 697411256
- 무작위로 선택된 값



확인 응답

클라이언트 순서 번호 확인:

- 확인 응답 번호: 3588415413
- (클라이언트 순서 번호 + 1)



플래그 설정

두 개의 플래그 활성화:

- SYN=1 (연결 요청)
- ACK=1 (응답 확인)

```
▼ Transmission Control Protocol, Src Port: 49859, Dst Port: 80, Seq: 0, Len: 0
  Source Port: 49859
  Destination Port: 80
  [Stream index: 0]
  [Conversation completeness: Incomplete, ESTABLISHED (7)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 3588415412
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  1010 .... = Header Length: 40 bytes (10)
  > Flags: 0x002 (SYN)
    Window: 8192
    [Calculated window size: 8192]
    Checksum: 0x303f [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  > Options: (20 bytes), Maximum segment size, No-Operation (NOP), Window scale, SACK permitted, T
  > [Timestamps]
```

TCP 패킷 분석 – 3단계(ACK)

세 번째 패킷은 클라이언트가 서버의 SYN+ACK에 대한 응답으로 ACK를 보내는 패킷입니다. 이 단계가 완료되면 양방향 연결이 수립됩니다.

포트 번호

첫 번째 패킷과 동일:

- 송신지 포트: 49859 (클라이언트)
- 수신지 포트: 80 (서버)

순서 번호

클라이언트의 순서 번호 유지:

- 순서 번호: 3588415413
- (SYN 패킷의 순서 번호 + 1)

확인 응답 번호

서버의 순서 번호 확인:

- 확인 응답 번호: 697411257
- (서버 순서 번호 + 1)

플래그

하나의 플래그만 활성화:

- ACK=1 (응답 확인)
- SYN=0 (더 이상 연결 설정 필요 없음)

Transmission Control Protocol, Src Port: 80, Dst Port: 49859, Seq: 0, Ack: 1, Len: 0

Source Port: 80

Destination Port: 49859

[Stream index: 0]

[Conversation completeness: Incomplete, ESTABLISHED (7)]

[TCP Segment Len: 0]

Sequence Number: 0 (relative sequence number)

Sequence Number (raw): 697411256

[Next Sequence Number: 1 (relative sequence number)]

Acknowledgment Number: 1 (relative ack number)

Acknowledgment number (raw): 3588415413

1010 = Header Length: 40 bytes (10)

Flags: 0x012 (SYN, ACK)

Window: 5792

[Calculated window size: 5792]

Checksum: 0x64df [unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

> Options: (20 bytes), Maximum segment size, No-Operation (NOP), No-Operation (NOP), Timestamps, No-Operation (NOP),

> [Timestamps]

> [SEQ/ACK analysis]

TCP 연결 수립 과정 요약

1 — 1단계: SYN 패킷

클라이언트 → 서버

- SYN 플래그 설정
- 초기 순서 번호(ISN) 전송
- 예: 순서 번호 = 3588415412

2 — 2단계: SYN+ACK 패킷

서버 → 클라이언트

- SYN, ACK 플래그 설정
- 서버의 ISN 전송
- 클라이언트 ISN+1 확인
- 예: 순서 번호 = 697411256, 확인 응답 번호 = 3588415413

3 — 3단계: ACK 패킷

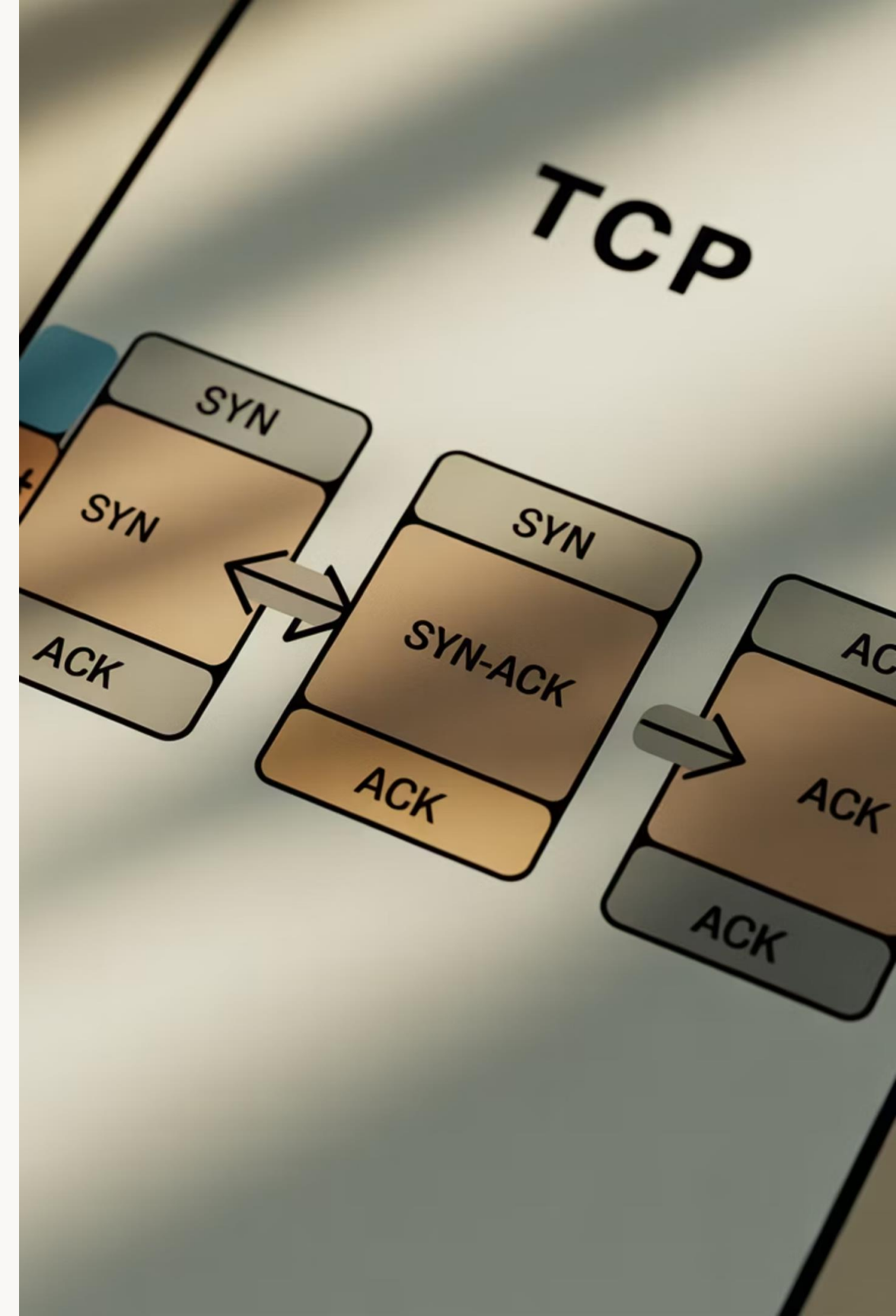
클라이언트 → 서버

- ACK 플래그 설정
- 서버의 ISN+1 확인
- 예: 순서 번호 = 3588415413, 확인 응답 번호 = 697411257

4 — 연결 설정 완료

양방향 통신 준비

- 데이터 전송 시작 가능
- 각 호스트는 상대방의 순서 번호를 알고 있음
- 신뢰성 있는 전송 채널 수립



TCP 연결 종료 과정

FIN 전송

종료를 원하는 측에서 FIN 플래그가 설정된 세그먼트를 전송

FIN에 대한 ACK

상대방은 FIN에 대한 응답으로 ACK를 전송

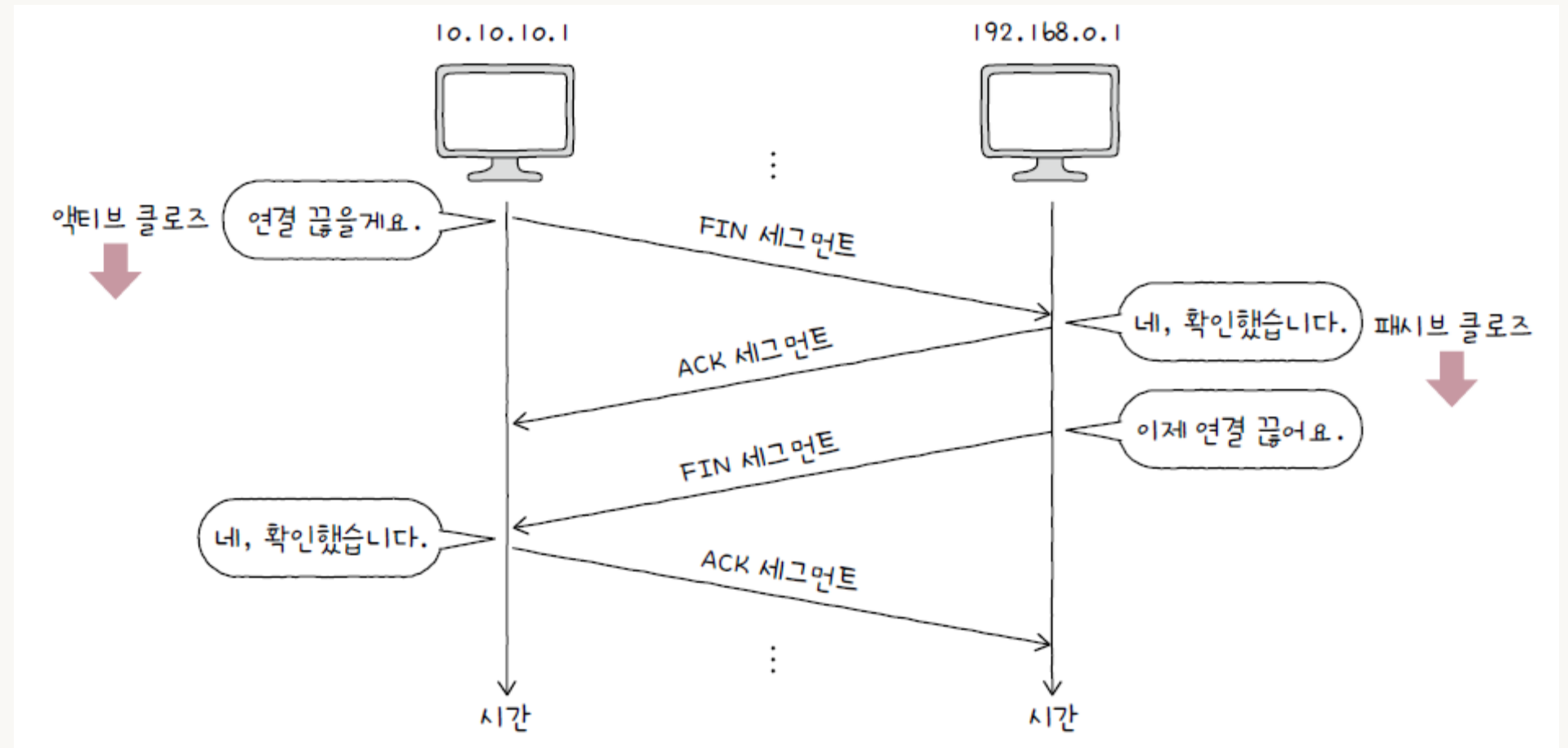
상대방의 FIN

상대방도 연결 종료를 위해 FIN 플래그가 설정된 세그먼트 전송

마지막 ACK

마지막 FIN에 대한 ACK를 전송하면
연결 종료 완료

TCP의 연결 종료 과정은 통신을 주고받는 호스트가 서로 FIN과 ACK를 주고받으며 이루어집니다.



TCP 연결 종료 - 첫 번째 패킷

첫 번째 패킷 분석

10.10.10.1이 192.168.0.1로 FIN 비트가 설정된 세그먼트를 전송합니다. 이는 연결 종료 요청과 같으며, 액티브 클로즈(Active Close)의 과정입니다.

- ❏ 액티브 클로즈란 연결 종료를 먼저 시작하는 측의 동작을 의미합니다. 반대로 연결 종료 요청을 받는 측은 패시브 클로즈(Passive Close)라고 합니다.

```
Sequence Number: 1      (relative sequence number)
Sequence Number (raw): 290236744
[Next Sequence Number: 2      (relative sequence number)]
Acknowledgment Number: 1      (relative ack number)
Acknowledgment number (raw): 951058419
0101 ..... = Header Length: 20 bytes (5)
v Flags: 0x011 (FIN, ACK)
  000. .... = Reserved: Not set
  ...0 .... = Accurate ECN: Not set
  .... 0... = Congestion Window Reduced: Not set
  .... .0.. = ECN-Echo: Not set
  .... ..0. = Urgent: Not set
  .... ...1 .... = Acknowledgment: Set
  .... .... 0... = Push: Not set
  .... .... .0.. = Reset: Not set
  .... .... ..0. = Syn: Not set
> .... .... ...1 = Fin: Set
```

TCP 연결 종료 - 두 번째 패킷

```
Sequence Number: 1      (relative sequence number)
Sequence Number (raw): 951058419
[Next Sequence Number: 1      (relative sequence number)]
Acknowledgment Number: 2      (relative ack number)
Acknowledgment number (raw): 290236745
0101 ..... = Header Length: 20 bytes (5)
~ Flags: 0x010 (ACK)
  000. .... = Reserved: Not set
  ...0 .... = Accurate ECN: Not set
  .... 0... = Congestion Window Reduced: Not set
  .... .0.. = ECN-Echo: Not set
  .... ..0. = Urgent: Not set
  .... ...1 = Acknowledgment: Set
  .... .... 0... = Push: Not set
  .... .... .0.. = Reset: Not set
  .... .... ..0. = Syn: Not set
  .... .... ...0 = Fin: Not set
```

두 번째 패킷 분석

192.168.0.1이 첫 번째 패킷에 대한 응답으로 ACK 비트가 설정된 세그먼트를 전송합니다.

ACK 응답

FIN 요청을 받았음을 알리는 응답 패킷입니다.

CLOSE_WAIT 상태

이 시점에서 192.168.0.1은 CLOSE_WAIT 상태로 전환됩니다.

TCP 연결 종료 - 세 번째 패킷

세 번째 패킷 분석

어느 정도의 시간이 흐른 뒤 192.168.0.1은 FIN 비트가 설정된 세 번째 세그먼트를 보냅니다.

❗ 이 시점에서 192.168.0.1은 자신의 데이터 전송을 모두 완료했으며, 이제 연결을 종료하고 싶다는 의사를 표현합니다.

시간 간격은 애플리케이션의 처리 시간에 따라 달라질 수 있으며, 모든 데이터 전송이 완료되었을 때 FIN 세그먼트를 전송합니다.

```
Sequence Number: 1      (relative sequence number)
Sequence Number (raw): 951058419
[Next Sequence Number: 2      (relative sequence number)]
Acknowledgment Number: 2      (relative ack number)
Acknowledgment number (raw): 290236745
0101 ..... = Header Length: 20 bytes (5)
v Flags: 0x011 (FIN, ACK)
  000. .... .... = Reserved: Not set
  ...0 .... .... = Accurate ECN: Not set
  .... 0... .... = Congestion Window Reduced: Not set
  .... .0.. .... = ECN-Echo: Not set
  .... ..0. .... = Urgent: Not set
  .... ...1 .... = Acknowledgment: Set
  .... .... 0... = Push: Not set
  .... .... .0.. = Reset: Not set
  .... .... ..0. = Syn: Not set
> .... .... ...1 = Fin: Set
```

TCP 연결 종료 - 네 번째 패킷

```
Sequence Number: 2      (relative sequence number)
Sequence Number (raw): 290236745
[Next Sequence Number: 2      (relative sequence number)]
Acknowledgment Number: 2      (relative ack number)
Acknowledgment number (raw): 951058420
0101 ..... = Header Length: 20 bytes (5)
~ Flags: 0x010 (ACK)
  000. .... = Reserved: Not set
  ...0 .... = Accurate ECN: Not set
  .... 0... = Congestion Window Reduced: Not set
  .... .0.. = ECN-Echo: Not set
  .... ..0. = Urgent: Not set
  .... ...1 .... = Acknowledgment: Set
  .... .... 0... = Push: Not set
  .... .... .0.. = Reset: Not set
  .... .... ..0. = Syn: Not set
  .... .... ...0 = Fin: Not set
```

네 번째 패킷 분석

10.10.10.1이 192.168.0.1에게 ACK 비트가 설정된 세그먼트를 보내면 연결 종료 과정이 완료됩니다.



1

즉시 종료

192.168.0.1의 경우 네 번째 패킷을 받으면 즉시 연결이 종료됩니다.



2

TIME_WAIT 상태

10.10.10.1의 경우 네 번째 패킷을 전송한 후에 일정 시간(TIME_WAIT)을 대기한 뒤 연결이 종료됩니다.

8.0.1
10.1
8.0.1
10.1
10.1

HTTP 프로토콜 분석

HTTP(HyperText Transfer Protocol)는 웹 브라우저와 웹 서버 간의 통신을 위한 프로토콜입니다. 와이어샤크를 통해 HTTP 요청과 응답 메시지를 분석할 수 있습니다.

- HTTP의 요청과 응답 헤더의 분석은 웹 브라우저의 개발자 도구를 이용해도 쉽게 할 수 있습니다. 하지만 와이어샤크를 사용하면 더 낮은 계층의 정보까지 함께 분석할 수 있습니다.



HTTP 요청

클라이언트가 서버에 보내는 메시지로, 메서드, URL, 헤더, 본문으로 구성



HTTP 응답

서버가 클라이언트에 보내는 메시지로, 상태 코드, 헤더, 본문으로 구성

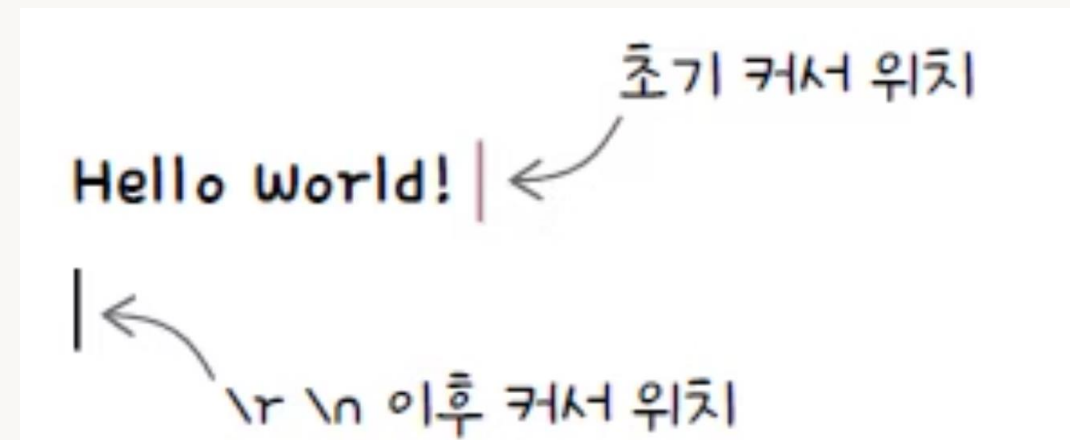
HTTP 요청분석

```
▼ Hypertext Transfer Protocol
  ▼ GET / HTTP/1.1\r\n
    > [Expert Info (Chat/Sequence): GET / HTTP/1.1\r\n]
      Request Method: GET
      Request URI: /
      Request Version: HTTP/1.1
      Host: info.cern.ch\r\n
      Connection: keep-alive\r\n
      Upgrade-Insecure-Requests: 1\r\n
      User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/121.0.0.0 Safari/537.36 Edg/121.0.0.0\r\n
      Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7\r\n
      Accept-Encoding: gzip, deflate\r\n
      Accept-Language: ko,en;q=0.9,en-US;q=0.8\r\n
      \r\n
      [Full request URI: http://info.cern.ch/]
      [HTTP request 1/1]
      [Response in frame: 2]
```

HTTP 요청 메시지 구조

1번 패킷은 웹 브라우저에서 `http://info.cern.ch/`를 입력했을 때 캡처된 패킷입니다. 이는 HTTP GET 메서드를 사용하여 `info.cern.ch` 서버의 루트 경로(/)를 요청합니다.

- ① `\r`과 `\n`은 각각 캐리지 리턴(Carriage Return)과 라인 피드(Line Feed)를 의미합니다. 이 두 문자를 함께 사용하면 커서의 위치가 행의 앞으로 이동한 뒤 다음 행으로 이동하게 되어 행바꿈(줄바꿈)을 표현합니다.



HTTP 응답 분석

```
✓ Hypertext Transfer Protocol
  ✓ HTTP/1.1 200 OK\r\n
    > [Expert Info (Chat/Sequence): HTTP/1.1 200 OK\r\n]
      Response Version: HTTP/1.1
      Status Code: 200
      [Status Code Description: OK]
      Response Phrase: OK
      Date: Wed, 07 Feb 2024 16:16:53 GMT\r\n
      Server: Apache\r\n
      Last-Modified: Wed, 05 Feb 2014 16:00:31 GMT\r\n
      ETag: "286-4f1aadb3105c0"\r\n
      Accept-Ranges: bytes\r\n
    > Content-Length: 646\r\n
      Connection: close\r\n
      Content-Type: text/html\r\n
      \r\n
      [HTTP response 1/1]
      [Time since request: 0.224151000 seconds]
      [Request in frame: 1]
      [Request URI: http://info.cern.ch/]
      File Data: 646 bytes
    > Line-based text data: text/html (13 lines)
```

HTTP 응답 메시지 구조

2번 패킷은 앞의 요청에 대한 HTTP 응답입니다. 응답 코드(Status Code)가 200으로, 요청이 성공적으로 처리되었음을 의미합니다.

상태 라인

HTTP/1.1 200 OK – 프로토콜 버전, 상태 코드, 상태 메시지

헤더 필드

Content-Type: text/html – 응답 본문이 HTML 문서임을 나타냄

응답 본문

실제 HTML 콘텐츠가 포함됨

HTTP 응답

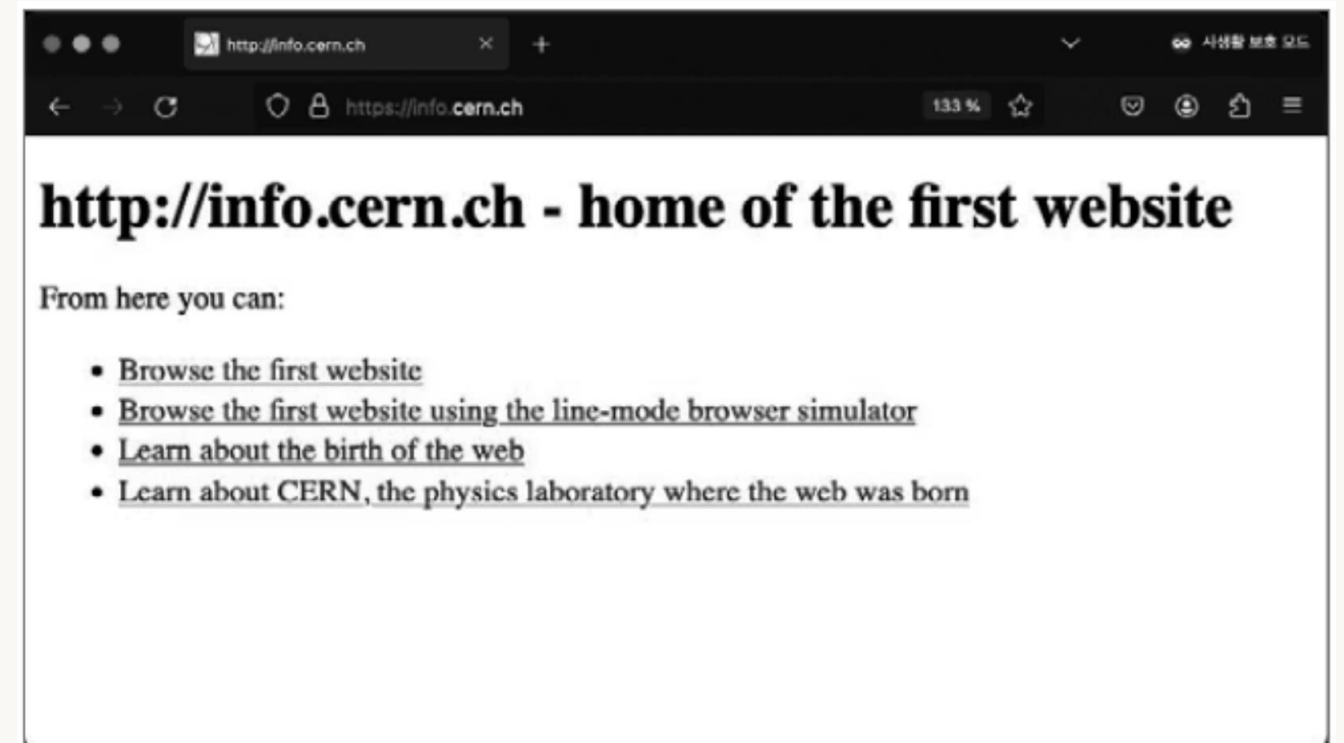
- 실제 웹페이지

```
Line-based text data: text/html (13 lines)
<html><head></head><body><header>\n
<title>http://info.cern.ch</title>\n
</header>\n
\n
<h1>http://info.cern.ch - home of the first website</h1>\n
<p>From here you can:</p>\n
<ul>\n
<li><a href="http://info.cern.ch/hypertext/WWW/TheProject.html">Browse the first website</a></li>\n
<li><a href="http://line-mode.cern.ch/www/hypertext/WWW/TheProject.html">Browse the first website using the line-mode browser simulator</a></li>\n
<li><a href="http://home.web.cern.ch/topics/birth-web">Learn about the birth of the web</a></li>\n
<li><a href="http://home.web.cern.ch/about">Learn about CERN, the physics laboratory where the web was born</a></li>\n
</ul>\n
</body></html>\n
```

응답 HTML과 렌더링된 웹페이지

HTTP 응답 메시지에 포함된 HTML 코드가 웹 브라우저에서 어떻게 렌더링되는지 확인할 수 있습니다. 이것은 세계 최초의 웹사이트인 CERN의 정보 페이지입니다.

- 이 페이지는 웹의 창시자인 팀 버너스-리(Tim Berners-Lee)가 만든 세계 최초의 웹사이트 중 하나로, 웹의 역사적 의미를 가지고 있습니다.



HTTP 링크 클릭 시 분석

3번 패킷은 앞의 화면에서 "Browse the first website"라는 링크를 클릭했을 때 캡처된 패킷입니다.

요청 경로

요청을 보낸 경로는 /hypertext/WWW/TheProject.html로, 이전 페이지에서 링크로 연결된 페이지입니다.

HTTP 메서드

요청에 사용된 HTTP 메서드는 GET으로, 웹 페이지를 조회하는 용도로 사용됩니다.

Referer 헤더

Referer 헤더를 통해 이전에 요청을 보낸 자원이 <http://info.cern.ch>임을 확인할 수 있습니다.

```
▼ Hypertext Transfer Protocol
  ▼ GET /hypertext/WWW/TheProject.html HTTP/1.1\r\n
    > [Expert Info (Chat/Sequence): GET /hypertext/WWW/TheProject.html HTTP/1.1\r\n]
      Request Method: GET
      Request URI: /hypertext/WWW/TheProject.html
      Request Version: HTTP/1.1
      Host: info.cern.ch\r\n
      Connection: keep-alive\r\n
      Upgrade-Insecure-Requests: 1\r\n
      User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/121.0.0.0 Safari/537.36 Edg/121.0.0.0\r\n
      Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7\r\n
      Referer: http://info.cern.ch/\r\n
      Accept-Encoding: gzip, deflate\r\n
      Accept-Language: ko,en;q=0.9,en-US;q=0.8\r\n
```

세계 최초 웹사이트 응답 분석

5번 패킷 분석 5번 패킷은 3번 패킷에 대한 HTTP 응답입니다. 2번 패킷과 마찬가지로 HTML 문서를 응답받았습니다.

응답받은 HTML 문서는 '세계 최초 웹 사이트'의 모습을 보여줍니다. 이 웹사이트는 1991년에 만들어진 것으로, 웹의 초기 역사를 보여주는 중요한 자료입니다.



- ✔ 와이어샤크를 통해 HTTP 프로토콜의 요청과 응답 패킷을 분석함으로써, 웹 통신의 기본 원리를 이해할 수 있습니다. 이러한 분석은 웹 개발자와 네트워크 관리자에게 중요한 기술입니다.